

Docker Compose

para Devs

Do conceito a prática com analogias do nosso mundo

De um a muitos containers

3 Exercícios Práticos

Cheat Sheet

Sumário

01	O que é Docker Compose?	2
02	Pré-requisito: relembre o Docker	3
03	Para que serve o Docker Compose?	4
04	Instalando o Docker Compose	5
05	Conceitos essenciais	6
06	Exercício 1. Subindo seu primeiro Compose	8
07	Exercício 2. App com banco de dados	9
08	Exercício 3. Ambiente completo de dev	11
09	Cheat Sheet	13
10	Próximos passos	14

Como usar este mini book

Leia do começo ao fim na primeira vez.

Os capítulos foram pensados em sequência: cada um prepara o terreno para o seguinte. Se você pular direto para os exercícios sem ler os conceitos, pode ficar perdida em alguns comandos.

Volte sempre que precisar.

O Cheat Sheet da página 13 foi feito para ficar aberto enquanto você pratica. Deixe ele no segundo monitor ou impresso do lado do computador.

Faca os exercícios de verdade.

Ler sobre Docker ensina a teoria. Rodar os comandos é que fixa de verdade. Abra o terminal e execute cada passo. Não tem atalho aqui.

01 • O que é Docker Compose?

Você já aprendeu a subir um container com o Docker. Mas e quando a sua aplicação precisa de mais de um? Uma API, um banco de dados, um serviço de cache. Gerenciar tudo isso na mão, container por container, vira um pesadelo muito rápido. É aí que entra o Docker Compose.

O Docker Compose é uma ferramenta que permite definir e executar aplicações com **múltiplos containers** usando um único arquivo de configuração: o **docker-compose.yml**. Com um único comando você sobe tudo. Com outro você derruba tudo. Simples assim.

>> Imagina que você vai organizar um jantar temático para suas amigas. Em vez de ligar para cada fornecedor separado. a doceira, a florista, a DJ. você contrata uma assessoria de eventos que cuida de tudo de uma vez. Você só diz o que quer no seu roteiro do evento e ela garante que cada peça esteja no lugar certo, na hora certa. O docker-compose.yml é esse roteiro e o Docker Compose é a assessoria.

[DICA] Mas eu preciso saber Docker antes?

Sim! O Docker Compose é construído em cima do Docker. Se você ainda não leu o Volume 03 da série, recomendamos muito que comece por lá. Ele cobre containers, imagens, Dockerfile e os comandos essenciais que você vai usar bastante aqui.

02 • Pré-requisito: relembre o Docker

Antes de mergulhar no Compose, vale lembrar os conceitos do Docker que você vai encontrar o tempo todo aqui. Se você leu o Volume 03, isso vai ser uma revisão rápida. Se é a primeira vez, recomendamos começar por lá.

[DICA] Volume 03. Docker para Devs

O Volume 03 da Série Deploy do Zero cobre tudo que você precisa saber antes de começar aqui: o que é um container, como escrever um Dockerfile, como usar o Docker Hub e os comandos essenciais. Leitura obrigatória antes deste volume!

O que você precisa lembrar

- **Container:** uma instância em execução de uma imagem Docker. É isolado, leve e descartável.
- **Image:** o molde do container. imutável e reutilizável.
- **Dockerfile:** o arquivo que define como sua imagem é construída.
- **docker build:** constrói uma imagem a partir do Dockerfile.
- **docker run:** cria e inicia um container a partir de uma imagem.
- **Porta (-p):** mapeia uma porta do container para a sua máquina.
- **Volume:** persiste dados do container no disco da máquina.

>> Se o Docker é cada peça do look. o vestido, os brincos, a bolsa e os sapatos. então o Docker Compose é o aplicativo de moda que te ajuda a montar e salvar o look completo de uma vez, com todas as peças combinando entre si, sem precisar escolher cada uma na mão toda vez que quiser usar.

Docker vs Docker Compose

Docker sozinho	Com Docker Compose
Um container por vez	Todos os containers juntos
Comandos longos e repetitivos	Um único arquivo de configuração
Redes manuais entre containers	Rede automática entre serviços
Reinício manual de cada serviço	<code>docker compose up</code> sobe tudo
Difícil de compartilhar config	Arquivo versionado no Git

03 · Para que serve o Docker Compose?

Beleza, você já sabe o que é. Mas onde você vai encontrar o Docker Compose no seu dia a dia? Vem que eu te mostro os casos de uso mais comuns.

Ambiente de desenvolvimento local

Quer rodar sua aplicação completa na sua máquina com um único comando? Com o Compose você define todos os serviços. API, banco, cache, fila. num arquivo só e sobe tudo junto. A sua colega clona o repositório, roda **docker compose up** e em segundos está com o mesmo ambiente que você.

Integração entre serviços

No Compose, cada serviço consegue falar com outro usando apenas o nome do serviço como hostname. A API chama o banco pelo nome **db**, o serviço de autenticação chama o Redis pelo nome **redis**. Sem precisar saber IPs, sem configuração extra.

>> *Pensa na sua mesa de jantar. Cada cadeira tem seu lugar, cada prato tem o seu espaço, e todo mundo sabe onde se sentar. O Compose organiza seus containers assim: cada um no seu lugar, todos se comunicando direitinho, sem ninguém se perder.*

Ambientes de teste e CI/CD

Pipelines de CI usam o Docker Compose para subir o banco de dados e os serviços dependentes antes de rodar os testes. Quando os testes terminam, tudo é derrubado e descartado. Limpo, reproduzível e sem efeitos colaterais.

Prototipagem rápida

Quer testar uma nova tecnologia? MongoDB, RabbitMQ, Elasticsearch? Escreve um Compose com aquele serviço, experimenta e descarta. Sem instalar nada na sua máquina de verdade.

[STAT] Compose no mercado

O Docker Compose é padrão em praticamente todo time de desenvolvimento moderno. Saber ler, escrever e depurar um docker-compose.yml é habilidade essencial para vagas backend, DevOps e cloud. É um dos primeiros arquivos que você vai encontrar ao entrar num projeto novo.

04 · Instalando o Docker Compose

A boa notícia é que se você já tem o Docker instalado, provavelmente já tem o Docker Compose também! A partir do Docker Desktop no Windows e Mac, e das versões recentes do Docker Engine no Linux, o Compose já vem embutido como plugin.

Windows e Mac. Docker Desktop

O Docker Desktop já inclui o Compose. Nenhuma instalação extra necessária. Para confirmar, abra o terminal e execute:

```
Terminal
docker compose version
# Esperado: Docker Compose version v2.x.x
```

Linux. verificar e instalar

No Linux, o Compose pode precisar ser instalado separadamente se você usar o Docker Engine puro:

```
Terminal Linux
# Verificar se já está instalado
docker compose version
# Se não estiver, instalar como plugin
sudo apt-get update
sudo apt-get install docker-compose-plugin -y
# Confirmar instalação
docker compose version
```

[!] docker-compose vs docker compose

Existe o comando legado docker-compose, que é a versão antiga V1, e o atual docker compose sem hífen, que é a versão V2 recomendada. A versão V2 é a recomendada hoje. Neste livro usamos sempre o docker compose sem hífen.

Estrutura básica do arquivo

O arquivo docker-compose.yml vive na raiz do seu projeto e segue esta estrutura:

```
docker-compose.yml. estrutura base
version: '3.8'           # versao do formato Compose
services:                # lista de containers
  nome-do-servico:
    image: nome-da-imagem
    ports:
      - 'host:container'
    environment:
      - VARIAVEL=valor
    volumes:
      - ./local:container
    depends_on:
      - outro-servico
volumes:                 # volumes nomeados
  nome-do-volume:
networks:                # redes customizadas opcional
  nome-da-rede:
```

05 · Conceitos essenciais

O Docker Compose tem vocabulário próprio. Vamos alinhar os termos que você vai ver em todo arquivo yaml e em toda discussão de arquitetura com o time.

Services (Serviços)

Definição: Cada container definido no docker-compose.yml é um serviço. Cada serviço tem nome, imagem ou build, portas, variáveis e dependências.

>> Cada integrante da banda tem seu instrumento, seu microfone e seu lugar no palco. O Compose define o lugar de cada um antes do show começar.

Networks (Redes)

Definição: O Compose cria automaticamente uma rede interna onde todos os serviços se enxergam pelo nome. Você também pode criar redes customizadas para isolar grupos de serviços.

>> É o grupo de WhatsApp do evento. Todo mundo que está no grupo consegue se comunicar. Quem não está no grupo não recebe as mensagens.

Volumes

Definição: Mecanismo para persistir dados além do ciclo de vida do container. Pode ser um volume nomeado, gerenciado pelo Docker, ou um bind mount que é uma pasta da sua máquina.

>> A bolsa eterna que você leva pra todo evento. O que você guardar nela continua lá mesmo que o container seja recriado do zero.

depends_on

Definição: Define a ordem de inicialização dos serviços. Um serviço com `depends_on` só sobe depois que os serviços listados já estiverem rodando.

>> É como a ordem de chegada pra montar o palco: primeiro o técnico de som, depois a iluminação, por último os músicos. O Compose respeita essa ordem.

Environment (Variáveis de ambiente)

Definição: Forma de passar configurações dinâmicas para o container como senhas, URLs, portas e tokens. Nunca coloque segredos escritos diretamente no Compose!

>> O roteiro personalizado que você passa para cada fornecedor do evento: o nome do cliente, o endereço, o horário. Cada um recebe as informações que precisa.

[!] `depends_on` não espera o serviço estar pronto

O `depends_on` garante que o container foi iniciado, mas não que está realmente pronto para receber conexões. Para aguardar o banco estar disponível de verdade, use healthchecks. Você vai ver isso aplicado no Exercício 3!

Profiles

Definição: Permite marcar serviços opcionais que só sobem quando você pede. É ótimo para ferramentas de debug, serviços de monitoramento ou dependências de teste.

>> O fotógrafo que só vem quando você contrata o pacote premium. Na festa básica ele não aparece, mas quando você quer o registro completo, ele está lá.

[DICA] Resumo rápido antes de praticar

Services são os containers. Networks são as redes de comunicação. Volumes persistem os dados. O `depends_on` define a ordem de subida. O `Environment` passa as configurações. E os Profiles ativam serviços opcionais. Com isso você já consegue ler qualquer `docker-compose.yml` real!

Exercício 1 — Subindo seu primeiro Compose

INICIANTE

Objetivo: Criar e executar seu primeiro `docker-compose.yml` com um serviço simples.

Tempo estimado: 15 minutos

O que você vai aprender

- Como escrever um **`docker-compose.yml`** do zero
- Como usar **`docker compose up`** e **`docker compose down`**
- Como ver logs com **`docker compose logs`**
- Como verificar o status dos serviços

Passo a passo

1. Crie uma pasta e entre nela:

```
mkdir meu-primeiro-compose
cd meu-primeiro-compose
```

2. Crie o arquivo **docker-compose.yml**:

```
docker-compose.yml
version: '3.8'
services:
  web:
    image: nginx:alpine
    ports:
      - '8080:80'
    volumes:
      - ./html:/usr/share/nginx/html
```

Terminal

```
# Crie a pasta html e um arquivo index
mkdir html
echo "<h1>Meu primeiro Compose!</h1>" > html/index.html
```

3. Suba o serviço:

```
docker compose up -d
# Verificar que está rodando
docker compose ps
```

Acesse **http://localhost:8080** e veja sua página aparecendo.

4. Explore os comandos essenciais do Compose:

```
docker compose logs          # ver logs de todos os serviços
docker compose logs -f web   # seguir logs do serviço web
docker compose ps            # status dos serviços
docker compose stop          # para sem remover
docker compose down          # para e remove containers e redes
docker compose down -v       # para tudo e remove os volumes
```

[OK] Resultado esperado

O serviço Nginx subiu, você acessou pelo browser e praticou os principais comandos do Compose. Se apareceu 'Meu primeiro Compose!' na tela. missão cumprida!

[OK] Resultado esperado. Exercício 1

O serviço Nginx subiu, você acessou pelo browser e praticou os principais comandos do Compose. Se apareceu 'Meu primeiro Compose!' na tela. missão cumprida!

Exercício 2 — App com Banco de Dados

INTERMEDIARIO

Objetivo: Orquestrar uma API Python conectada a um banco PostgreSQL usando Docker Compose.

Tempo estimado: 25 a 35 minutos

O que você vai aprender

- Como conectar dois serviços pela rede interna do Compose
- Como usar **depends_on** para garantir a ordem de inicialização
- Como usar volumes para persistir dados do banco
- Como passar variáveis de ambiente com segurança

Vamos criar uma API Flask que se conecta a um PostgreSQL, tudo orquestrado pelo Compose.

Passo a passo

1. Crie a estrutura do projeto:

```
mkdir app-com-banco && cd app-com-banco
mkdir api
```

2. Crie **api/app.py**:

api/app.py

```
# api/app.py
from flask import Flask, jsonify
import psycopg2, os
app = Flask(__name__)
@app.route('/')
def status():
    try:
        conn = psycopg2.connect(
            host=os.getenv('DB_HOST', 'db'),
            database=os.getenv('DB_NAME', 'appdb'),
            user=os.getenv('DB_USER', 'postgres'),
            password=os.getenv('DB_PASS', 'senha123')
        )
        conn.close()
        return jsonify({'status': 'conectada ao banco!'})
    except Exception as e:
        return jsonify({'erro': str(e)}), 500
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

3. Crie **api/requirements.txt** e **api/Dockerfile**:

api/requirements.txt

```
# requirements.txt
flask==3.0.0
psycopg2-binary==2.9.9
```

api/Dockerfile

```
# api/Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

4. Crie o **docker-compose.yml** na raiz:

```
docker-compose.yml
version: '3.8'
services:
  api:
    build: ./api
    ports:
      - '5000:5000'
    environment:
      - DB_HOST=db
      - DB_NAME=appdb
      - DB_USER=postgres
      - DB_PASS=senha123
    depends_on:
      - db
  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_DB=appdb
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=senha123
    volumes:
      - dados-db:/var/lib/postgresql/data
volumes:
  dados-db:
```

5. Suba tudo:

```
docker compose up -d
# Aguarde o banco inicializar e teste
curl http://localhost:5000
# Esperado: {status: conectada ao banco!}
```

>> A API é a garçonete e o banco é a cozinha. A garçonete só começa a atender quando a cozinha já está funcionando, e é isso que o `depends_on` garante. O nome `'db'` no código da API é o endereço interno da cozinha, que só funciona porque os dois estão na mesma rede do evento.

[OK] Resultado esperado

A API sobe, consegue se conectar ao PostgreSQL pelo nome `'db'` e retorna o status de conexão. Os dados do banco persistem no volume mesmo se você der `docker compose down` e `up` novamente.

[OK] Resultado esperado. Exercício 2

A API sobe, consegue se conectar ao PostgreSQL pelo nome 'db' e retorna o status de conexão. Os dados do banco persistem no volume mesmo se você der docker compose down e up novamente.

Exercício 3 — Ambiente Completo de Desenvolvimento

DESAFIO

Objetivo: Montar um ambiente de desenvolvimento completo com API, banco, cache Redis e painel de administração.

Tempo estimado: 40 a 50 minutos

[DICA] Spoiler do próximo volume!

Neste exercício você vai usar variáveis de ambiente via arquivo `.env`, `healthchecks` e `profiles`. São recursos avançados do Compose que a maioria dos devs só aprende na prática. Esse ambiente serve como base real para qualquer projeto.

O que você vai aprender

- Como usar **arquivo .env** para gerenciar variáveis
- Como configurar **healthchecks** para garantir serviços saudáveis
- Como usar **profiles** para serviços opcionais
- Como adicionar um painel **pgAdmin** para visualizar o banco

Passo a passo

1. Crie o arquivo **.env** na raiz do projeto (nunca suba esse arquivo pro Git!):

```
.env
# .env
POSTGRES_DB=appdb
POSTGRES_USER=postgres
POSTGRES_PASSWORD=senha_super_segura
REDIS_PASSWORD=redis_senha
PGADMIN_EMAIL=admin@deployada.com
PGADMIN_PASSWORD=admin123
```

2. Crie o **docker-compose.yml** completo:

```
docker-compose.yml
version: '3.8'
services:
  api:
    build: ./api
    ports:
      - '5000:5000'
    env_file: .env
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
  db:
    image: postgres:15-alpine
    env_file: .env
    volumes:
      - dados-db:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U $POSTGRES_USER']
      interval: 5s
      timeout: 5s
      retries: 5
  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
  pgadmin:
    image: dpage/pgadmin4
    profiles: ['tools']
    ports:
      - '5050:80'
    environment:
      - PGADMIN_DEFAULT_EMAIL=${PGADMIN_EMAIL}
      - PGADMIN_DEFAULT_PASSWORD=${PGADMIN_PASSWORD}
    depends_on:
      - db
volumes:
  dados-db:
```

3. Suba o ambiente principal e depois o painel de admin:

```
# Ambiente principal (sem o pgAdmin)
docker compose up -d
# Ativar o pgAdmin com profile tools
docker compose --profile tools up -d
# Acessar o pgAdmin em http://localhost:5050
```

4. Teste o healthcheck e veja a ordem de inicialização:

```
# Acompanhar em tempo real
docker compose logs -f
# Ver status de saúde dos containers
docker compose ps
```

>> Você organizou o evento completo: a garçone (API), a cozinha (banco), o bar (Redis cache) e até um fotógrafo profissional (pgAdmin) que só aparece quando você aciona o perfil 'tools'. O arquivo .env é o envelope lacrado com os segredos do evento que só a assessoria pode abrir.

[OK] Resultado esperado

O ambiente sobe com healthcheck garantindo que o banco está pronto antes da API conectar. O Redis está disponível para cache. O pgAdmin sobe apenas com o profile tools ativo. Você tem um ambiente de desenvolvimento profissional e reproduzível!

Erros comuns e como resolver

- **port is already allocated:** outra aplicação está usando a mesma porta. Troque o lado esquerdo do mapeamento, por exemplo '5001:5000'.
- **connection refused ao banco:** o depends_on não garante que o PostgreSQL está pronto. Use healthcheck com condition service_healthy.
- **variável de ambiente não encontrada:** confira se o arquivo .env está na mesma pasta que o docker-compose.yml e se o nome da variável está correto.
- **volume com dados antigos:** se o banco não inicializa com as novas configurações, faça docker compose down com a flag v para limpar o volume e recriar.

09 • Cheat Sheet de Comandos

Cola essa página nos favoritos. São os comandos do Docker Compose que você vai usar toda semana:

Subir e Derrubar	
<code>docker compose up</code>	Sobe todos os serviços em primeiro plano
<code>docker compose up -d</code>	Sobe todos os serviços em background
<code>docker compose up servico</code>	Sobe apenas um serviço específico
<code>docker compose down</code>	Para e remove containers e redes
<code>docker compose down -v</code>	Para tudo e remove todos os volumes
<code>docker compose stop</code>	Para os containers sem remover
<code>docker compose start</code>	Reinicia containers parados
<code>docker compose restart</code>	Reinicia todos os serviços
Monitorar e Inspeccionar	
<code>docker compose ps</code>	Lista status de todos os serviços
<code>docker compose logs</code>	Mostra logs de todos os serviços
<code>docker compose logs -f api</code>	Segue logs do serviço api
<code>docker compose top</code>	Lista processos rodando nos containers
<code>docker compose stats</code>	Monitora uso de CPU e memória
Build e Imagens	
<code>docker compose build</code>	Constrói todas as imagens com build
<code>docker compose build api</code>	Constrói apenas a imagem do serviço api
<code>docker compose pull</code>	Baixa as imagens mais recentes
<code>docker compose push</code>	Envia as imagens para o registry

Execução e Debug

<code>docker compose exec api bash</code>	Abre terminal no serviço api
<code>docker compose run api python -c ...</code>	Executa comando pontual no serviço
<code>docker compose config</code>	Valida e mostra o compose final
<code>docker compose --profile tools up</code>	Sobe serviços de um profile específico

10 • Próximos Passos

Parabéns por chegar até aqui! Você agora consegue orquestrar ambientes completos com Docker Compose. A série continua com temas que vão usar tudo que você aprendeu aqui:

Vol. 05. APIs REST com Python

Aprenda a construir APIs completas com Flask ou FastAPI, com boas práticas de REST, autenticação e documentação. Tudo dockerizado e com Compose, claro.

Vol. 06. AWS na Prática

Com Compose dominado localmente, vamos para a nuvem. ECS, ECR, S3 e Lambda são os próximos destinos para levar seus containers para produção na AWS.

Vol. 07. Mensageria: SQS & SNS

Aprenda a construir sistemas assíncronos com filas de mensagem. O Compose vai orquestrar sua API, o worker e o broker todos juntos.

Recursos para continuar estudando

- **Documentação oficial:** docs.docker.com/compose. referência completa e atualizada
- **Awesome Compose:** github.com/docker/awesome-compose. exemplos reais de Compose
- **Play with Docker:** labs.play-with-docker.com. pratique no browser sem instalar nada

- **Livro:** Docker Deep Dive de Nigel Poulton. capítulo dedicado ao Compose

O que você dominou neste volume

- O que é Docker Compose e por que ele é essencial no mercado
- Como estruturar um arquivo docker-compose.yml do zero
- Como conectar múltiplos serviços com redes internas
- Como usar depends_on, healthchecks e volumes
- Como gerenciar variáveis com arquivo .env
- Como usar profiles para serviços opcionais

Você chegou até aqui. Isso já te coloca na frente de muita gente.

Docker Compose não é só uma ferramenta. É a diferença entre um ambiente que só funciona na sua máquina e um que funciona em qualquer lugar do mundo.

Gostou desse mini book?

Compartilha com aquela amiga dev que tá começando a jornada!

Me marca no Instagram **@deployada** com o seu progresso.

Adoro acompanhar vocês evoluindo!

Instagram	LinkedIn	Newsletter
@deployada	linkedin.com/in/mgcarvalho	deployada

Continue sua jornada! >>

Vol. 04 • Docker Compose. orquestrando containers

Vol. 05 • APIs REST com Python

Vol. 06 • AWS na prática: S3, EC2 e Lambda

Vol. 07 • Mensageria: SQS & SNS

Vol. 08 • Observabilidade: Datadog & Splunk

"Toda grande dev um dia rodou seu primeiro container."

— @deployada

@deployada

Instagram • LinkedIn • Newsletter